

COL1000: Introduction to Programming

Lecture 2

**Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi**

Today's lecture

Basics of Python Language

1. How to output something in Python
2. How to store and manipulate data in Python
3. How to take inputs in Python
4. If-Else statements

How to run Python programs?

How to run Python programs?

1. Online Python interpreters

How to run Python programs?

1. Online Python interpreters

[https://www.programiz.com/python-programming/
online-compiler/](https://www.programiz.com/python-programming/online-compiler/)

How to run Python programs?

1. Online Python interpreters

[https://www.programiz.com/python-programming/
online-compiler/](https://www.programiz.com/python-programming/online-compiler/)

2. Use Python IDE on your computer, eg. IDLE.

How to run Python programs?

1. Online Python interpreters

<https://www.programiz.com/python-programming/online-compiler/>

2. Use Python IDE on your computer, eg. IDLE.

3. Use any editor and run code using Terminal.

How to run Python programs?

1. Online Python interpreters

<https://www.programiz.com/python-programming/online-compiler/>

2. Use Python IDE on your computer, eg. IDLE.

3. Use any editor and run code using Terminal.

Strongly advised to not autocomplete your code

First Python Program

First Python Program

Program

```
print("Hello World")
```

First Python Program

Program

```
print("Hello World")
```

Output

```
Hello World
```

First Python Program

Program

```
print("Hello")  
print("World")
```

First Python Program

Program

```
print("Hello")  
print("World")
```

Output

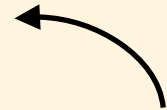
```
Hello  
World
```

First Python Program

Program

```
print("Hello")
```

```
print("World")
```



Note:

By default, each `print()` statement output is in a new line.

Output

```
Hello
```

```
World
```

First Python Program

Program

```
print("Hello", "World")
```


First Python Program

Program

```
print("Hello", "World")
```

Output

```
Hello World
```


First Python Program

Program

```
print("Hello", "World")
```

Note:

By default, Python inserts a space after each comma

Output

```
Hello World
```

First Python Program

Program

```
print("Hello", "World")
```

Note:

By default, Python inserts a space after each comma

Output

```
Hello World
```

Note:

`print("Hello", "World")` is different from `print("Hello,World")`

First Python Program

Program

```
print(hello)
```

First Python Program

Program

```
print(hello)
```

Output

```
ERROR!  
Traceback (most recent call last):  
  File "<main.py>", line 1, in <module>  
NameError: name 'hello' is not defined. Did you  
mean: 'help'?
```

First Python Program

Program

```
print(hello)
```

Output

```
ERROR!  
Traceback (most recent call last):  
  File "<main.py>", line 1, in <module>  
NameError: name 'hello' is not defined. Did you  
mean: 'help'?
```

Note: Mind the quotation marks !

Basics of Python Language

-  **1. How to output something in Python**
- 2. How to store and manipulate data in Python**
- 3. How to take inputs in Python**
- 4. If-Else statements**

Basics of Python Language

✓ 1. How to output something in Python

2. How to store and manipulate data in Python

3. How to take inputs in Python

4. If-Else statements

Data in Python

Data in Python

- Programs in computer store and manipulate **data**

Basic Data-types

Basic Data-types

- `int` – represent **integers**, example. 5

Basic Data-types

- `int` – represent **integers**, example. 5
- `float` – represent **decimal numbers**, example. 3.14

Basic Data-types

- `int` – represent **integers**, example. `5`
- `float` – represent **decimal numbers**, example. `3.14`
- `bool` – represent **Boolean** values `True` or `False`

Basic Data-types

- `int` – represent **integers**, example. `5`
- `float` – represent **decimal numbers**, example. `3.14`
- `bool` – represent **Boolean** values `True` or `False`
- `String` – represent **string** values, example. `"Hello world."`

Variable Declaration in Python

Variable Declaration in Python

- A variable in a program is a **named memory location** used for storing/referring data.

Variable Declaration in Python

- A variable in a program is a **named memory location** used for storing/referring data.
- **Syntax:** `variable_name = value`

Variable Declaration in Python

- A variable in a program is a **named memory location** used for storing/referring data.
- **Syntax:** variable_name = value

Example

```
age = 22                                # int
```

Variable Declaration in Python

- A variable in a program is a **named memory location** used for storing/referring data.
- **Syntax:** variable_name = value

Example

```
age = 22                                # int
weight = 50.2                          # float
```

Variable Declaration in Python

- A variable in a program is a **named memory location** used for storing/referring data.
- **Syntax:** variable_name = value

Example

```
age = 22                                # int
weight = 50.2                          # float
name = "ABC"                           # str
```

Variable Declaration in Python

- A variable in a program is a **named memory location** used for storing/referring data.
- **Syntax:** variable_name = value

Example

```
age = 22                # int
weight = 50.2           # float
name = "ABC"            # str
is_present = True       # bool
```

What happens on declaring a variable?

What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.

What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.

What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.
- This lets one access/change the value later in your code.

What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.
- This lets one access/change the value later in your code.

Memory:



What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.
- This lets one access/change the value later in your code.

Example

```
age = 22                                # int
```

Memory:



What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.
- This lets one access/change the value later in your code.

Example

```
age = 22                                     # int
```

Memory:



What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.
- This lets one access/change the value later in your code.

Example

```
age = 22                                # int
weight = 50.2                          # float
```

Memory:



What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.
- This lets one access/change the value later in your code.

Example

```
age = 22                                # int
weight = 50.2                          # float
```

Memory:



What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.
- This lets one access/change the value later in your code.

Example

```
age = 22                                # int
weight = 50.2                          # float
```

Memory:



Name of variable

What happens on declaring a variable?

- When a variable is created Python **reserves memory** to store the value.
- This lets one access/change the value later in your code.

Example

```
age = 22                                # int
weight = 50.2                          # float
```

Memory:



Arithmetic Operations

Arithmetic Operations

addition : +

Arithmetic Operations

addition : +

subtraction : -

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

Don't use "X" or "." for
multiplication

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Don't use "X" or "." for
multiplication

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

Don't use "X" or "." for
multiplication

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

remainder : %

Don't use "X" or "." for
multiplication

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

remainder : %

Don't use "X" or "." for
multiplication

Example:

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

remainder : %

Don't use "X" or "." for multiplication

Example:

a = 5

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

remainder : %

Don't use "X" or "." for multiplication

Example:

a = 5

b = 2

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

remainder : %

Don't use "X" or "." for multiplication

Example:

a = 5

b = 2

x = a + b

addition

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

remainder : %

Don't use "X" or "." for multiplication

Example:

a = 5

b = 2

x = a + b

addition

y = a * b

multiplication

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

remainder : %

Don't use "X" or "." for multiplication

Example:

a = 5

b = 2

x = a + b

addition

y = a * b

multiplication

z = x * (y % a) + 5

Arithmetic Operations

addition : +

subtraction : -

multiplication : *

True/Float division : /

Floor division : //

remainder : %

Don't use "X" or "." for multiplication

Example:

```
x = 5.0 / 2.0          # 2.5
y = 5 / 2              # 2.5 (again float)
z = 5 // 2             # 2   (answer is integer)

w = 5 % 2              # remainder
```


String Operations

String Operations

Concatenation : +

Copies of string : *

String Operations

Concatenation : +

Copies of string : *

Example

String Operations

Concatenation : `+`

Copies of string : `*`

Example

```
a = "with"
```

String Operations

Concatenation : `+`

Copies of string : `*`

Example

```
a = "with"
```

```
b = "out"
```

String Operations

Concatenation : `+`

Copies of string : `*`

Example

```
a = "with"
```

```
b = "out"
```

```
c = a + b
```

```
# without
```

String Operations

Concatenation : `+`

Copies of string : `*`

Example

```
a = "with"
```

```
b = "out"
```

```
c = a + b
```

```
# without
```

```
d = c * 2
```

```
# withoutwithout
```

String Operations

Concatenation : `+`

Copies of string : `*`

Example

```
a = "with"
```

```
b = "out"
```

```
c = a + b
```

```
# without
```

```
d = c * 2
```

```
# withoutwithout
```


String Operations

Concatenation : `+`

Copies of string : `*`

Example

```
a = "with"
```

```
b = "out"
```

```
c = a + b
```

```
# without
```

```
d = c * 2
```

```
# withoutwithout
```

```
x = "Hi\n" * 2 + "??"
```

```
# Hi
```

```
# Hi
```

```
# ??
```


Comparison Operation

Equal : `==`

Not Equal : `!=`

Greater than : `>`

Less than : `<`

Greater than or equal to : `>=`

Less than or equal to : `<=`

Example:

Comparison Operation

Equal : `==`

Not Equal : `!=`

Greater than : `>`

Less than : `<`

Greater than or equal to : `>=`

Less than or equal to : `<=`

Example:

`a = 3.5`

Comparison Operation

Equal : `==`

Not Equal : `!=`

Greater than : `>`

Less than : `<`

Greater than or equal to : `>=`

Less than or equal to : `<=`

Example:

`a = 3.5`

`b = 2.2`

Comparison Operation

Equal : `==`

Not Equal : `!=`

Greater than : `>`

Less than : `<`

Greater than or equal to : `>=`

Less than or equal to : `<=`

Example:

```
a = 3.5
```

```
b = 2.2
```

```
x = (a == b)           # stores if a is equal to b
```

Comparison Operation

Equal : `==`

Not Equal : `!=`

Greater than : `>`

Less than : `<`

Greater than or equal to : `>=`

Less than or equal to : `<=`

Example:

```
a = 3.5
```

```
b = 2.2
```

```
x = (a == b)           # stores if a is equal to b
```

```
y = (a != b)           # stores if a is not equal to b
```

Comparison Operation

Equal : `==`

Not Equal : `!=`

Greater than : `>`

Less than : `<`

Greater than or equal to : `>=`

Less than or equal to : `<=`

Example:

```
a = 3.5
```

```
b = 2.2
```

```
x = (a == b)           # stores if a is equal to b
```

```
y = (a != b)           # stores if a is not equal to b
```

```
z = (a > b)             # stores if a is greater than b
```

```
print("Hi"=="Hi")      # True
```

Logical Operations

Logical Operations

`and` : True only if all operands are True

`or` : False only if all operands are False

`not` : Converts True to False, and False to True

Logical Operations

`and` : True only if all operands are True

`or` : False only if all operands are False

`not` : Converts True to False, and False to True

Example:

Logical Operations

`and` : True only if all operands are True

`or` : False only if all operands are False

`not` : Converts True to False, and False to True

Example:

```
a = True
```

Logical Operations

`and` : True only if all operands are True

`or` : False only if all operands are False

`not` : Converts True to False, and False to True

Example:

```
a = True
```

```
b = False
```

Logical Operations

`and` : True only if all operands are True

`or` : False only if all operands are False

`not` : Converts True to False, and False to True

Example:

```
a = True
```

```
b = False
```

```
c = a and b
```

```
# c is False
```

Logical Operations

`and` : True only if all operands are True

`or` : False only if all operands are False

`not` : Converts True to False, and False to True

Example:

```
a = True
```

```
b = False
```

```
c = a and b           # c is False
```

```
d = a or b           # d is True
```

Logical Operations

`and` : True only if all operands are True

`or` : False only if all operands are False

`not` : Converts True to False, and False to True

Example:

```
a = True
```

```
b = False
```

```
c = a and b           # c is False
```

```
d = a or b           # d is True
```

```
e = not b           # e is True
```

Toy Program to compute area of circle

Toy Program to compute area of circle

Program

```
radius = 10
```


Toy Program to compute area of circle

Program

```
radius = 10
```

```
pi = 3.14
```

Toy Program to compute area of circle

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius
```

Toy Program to compute area of circle

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(area)
```

Toy Program to compute area of circle

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(area)
```

Output

Toy Program to compute area of circle

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(area)
```

Output

```
314
```

Toy Program to compute area of circle

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(area)
```

Output

314



Toy Program to compute area of circle

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(area)
```

Output

314



Output

Toy Program to compute area of circle

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(area)
```

Output

314



Output

314.0

Toy Program to compute area of circle

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(area)
```

Output

314

A large red 'X' mark is drawn over the output '314', indicating it is incorrect.

Output

314.0

A large green checkmark is drawn below the output '314.0', indicating it is correct.

Toy Program to compute area of circle

Program

```
radius = 10
pi = 3.14
area = pi * radius * radius
print(area)
```

Output

314



Output

314.0



When can we
do **operations** on
variables of **different**
data types?

Type Conversions in Python

Type Conversions in Python

Type conversion means **changing** one data type to another. These are of two types:

Type Conversions in Python

Type conversion means **changing** one data type to another. These are of two types:

- **Implicit conversion** – done automatically by Python

Type Conversions in Python

Type conversion means **changing** one data type to another. These are of two types:

- **Implicit conversion** – done automatically by Python
- **Explicit conversion** – done manually by the programmer

Type Conversions in Python

Type conversion means **changing** one data type to another. These are of two types:

- **Implicit conversion** – done automatically by Python
- **Explicit conversion** – done manually by the programmer

```
a = 3
```


Type Conversions in Python

Type conversion means **changing** one data type to another. These are of two types:

- **Implicit conversion** – done automatically by Python
- **Explicit conversion** – done manually by the programmer

```
a = 3
```

```
b = 4.5
```


Type Conversions in Python

Type conversion means **changing** one data type to another. These are of two types:

- **Implicit conversion** – done automatically by Python
- **Explicit conversion** – done manually by the programmer

```
a = 3
b = 4.5
c = a + b          # c becomes float (7.5) – implicit
```

Type Conversions in Python

Type conversion means **changing** one data type to another. These are of two types:

- **Implicit conversion** – done automatically by Python
- **Explicit conversion** – done manually by the programmer

```
a = 3
b = 4.5
c = a + b           # c becomes float (7.5) – implicit

x = "88"
```

Type Conversions in Python

Type conversion means **changing** one data type to another. These are of two types:

- **Implicit conversion** – done automatically by Python
- **Explicit conversion** – done manually by the programmer

```
a = 3
b = 4.5
c = a + b           # c becomes float (7.5) – implicit

x = "88"
y = int(x)          # y becomes int – explicit
```

When does Type Conversion work ?

When does Type Conversion work ?

- `int → float`

Example `float(5)`

When does Type Conversion work ?

- `int → float`

Example `float(5)`

- `float → int`

Example `int(5.6)`

When does Type Conversion work ?

- `int → float`

Example `float(5)`

- `float → int`

Example `int(5.6)`

- `str(numbers) → int/float`

Example `int("55")`

When does Type Conversion work ?

- `int → float`

Example `float(5)`

- `float → int`

Example `int(5.6)`

- `str(numbers) → int/float`

Example `int("55")`

- `bool → int`

Example `int(True)`

When does Type Conversion work ?

- `int → float`

Example `float(5)`

- `float → int`

Example `int(5.6)`

- `str(numbers) → int/float`

Example `int("55")`

- `bool → int`

Example `int(True)`

Examples that do not work

```
int("abc")
```

```
# ValueError
```

When does Type Conversion work ?

- `int → float`

Example `float(5)`

- `float → int`

Example `int(5.6)`

- `str(numbers) → int/float`

Example `int("55")`

- `bool → int`

Example `int(True)`

Examples that do not work

```
int("abc")
```

```
# ValueError
```

```
float("ten")
```

```
# ValueError
```

How to print area as integer..

How to print area as integer..

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(_____)
```

How to print area as integer..

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(_____)
```

Output

```
314
```

How to print area as integer..

Program

```
radius = 10  
pi = 3.14  
area = pi * radius * radius  
print(int(area))
```

Output

```
314
```

Basics of Python Language

-  **1. How to output something in Python**
-  **2. How to store and manipulate data in Python**
- 3. How to take inputs in Python**
- 4. If-Else statements**

Basics of Python Language

- ✓ 1. How to output something in Python
- ✓ 2. How to store and manipulate data in Python
- 3. How to take inputs in Python
- 4. If-Else statements

Input from User

Input from User

To take user input in Python:

Input from User

To take user input in Python:

- Use the built-in function `input()`

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is always a string, even if the user types a number.

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")
```

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + "!")
```

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + " !")  
  
age = input("Enter your age: ")
```


Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + "!")
```

```
age = input("Enter your age: ")  
print(age + 1)
```


Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + "!")
```

```
age = input("Enter your age: ")  
print(age + 1)
```



Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + "!")
```

```
age = input("Enter your age: ")  
print(age + 1)
```



TypeError: unsupported
operand types for +

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + "!")
```

```
age = input("Enter your age: ")  
print(age + 1)
```



TypeError: unsupported operand types for +

Would you get a TypeError if
the last line is
`print(age * 2)` ?

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is always a string, even if the user types a number.

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")
```

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + "!")
```

Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + "!")  
  
age = int(input("Enter your age: "))
```


Input from User

To take user input in Python:

- Use the built-in function `input()`
- The value returned by `input()` is **always a string**, even if the user types a number.

```
name = input("Enter your name: ")  
print("Hello, " + name + "!!")  
  
age = int(input("Enter your age: "))  
print(age + 1)
```


Exercise

Write a Python program that takes as input a **3-digit positive number**, and prints the digits of the number on separate lines (starting from ones digit).

Example:

Number: 314

Output:

4

1

3

Exercise

Write a Python program that takes as input a **3-digit positive number**, and prints the digits of the number on separate lines (starting from ones digit).

Example:

Number: 314

Output:

4

1

3

```
n = int(input("Enter the 4-digit number : "))

print("Output:")

digit1 = n%10
print(digit1)

n = n // 10          # removing the last digit
digit2 = n%10
print(digit2)

n = n // 10          # removing the last digit again
digit3 = n%10
print(digit3)
```

Exercise

Write a Python program that takes as input a **3-digit positive number**, and prints the digits of the number on separate lines (starting from ones digit).

Example:

Number: 314

Output:

4
1
3

```
n = int(input("Enter the 4-digit number : "))

print("Output:")

digit1 = n%10
print(digit1)

n = n // 10          # removing the last digit
digit2 = n%10
print(digit2)

n = n // 10          # removing the last digit again
digit3 = n%10
print(digit3)
```

Homework:

Code to reverse a 3-digit positive integer.

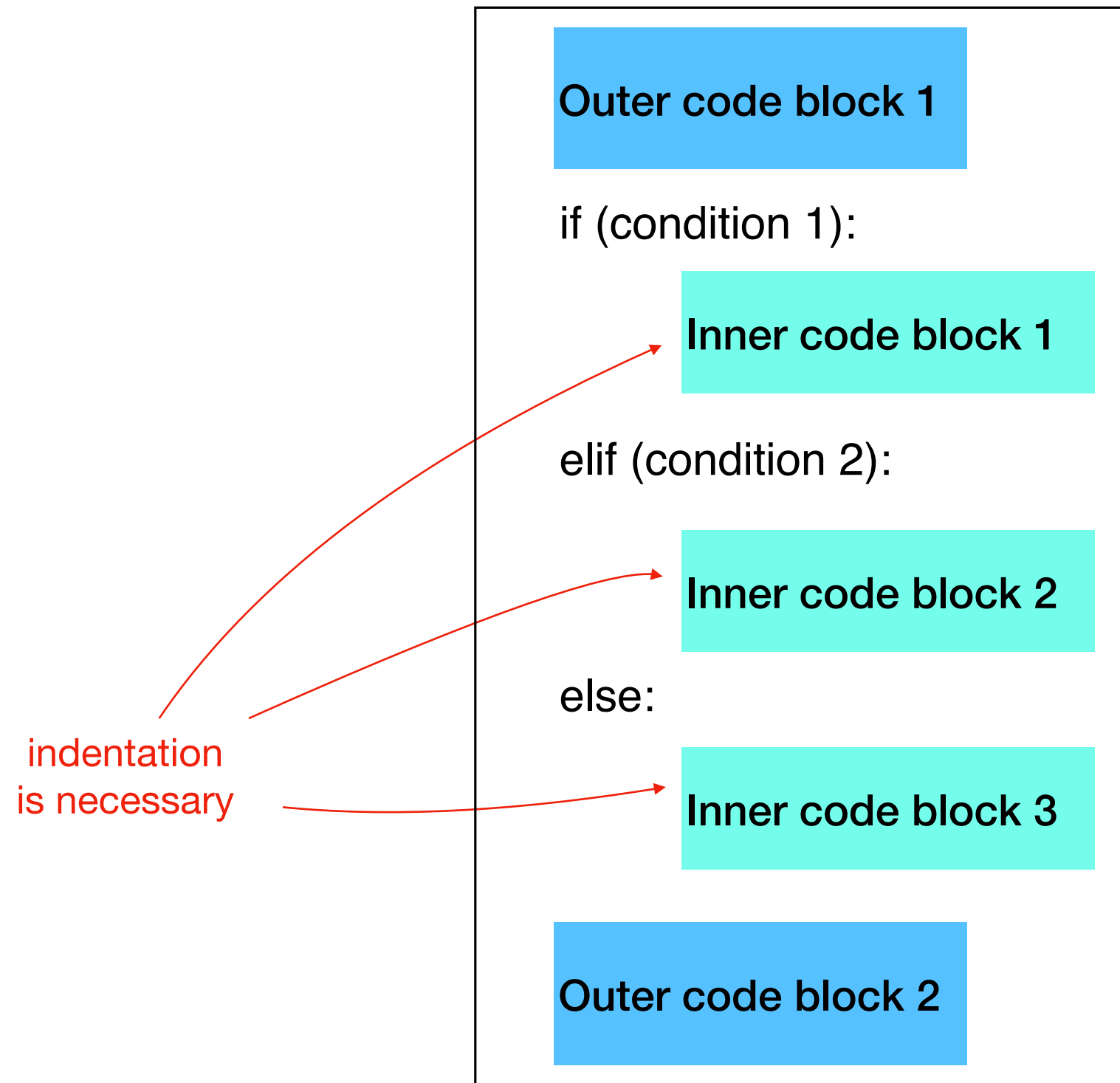
Basics of Python Language

-  **1. How to output something in Python**
-  **2. How to store and manipulate data in Python**
-  **3. How to take inputs in Python**
- 4. If-Else statements**

Basics of Python Language

- ✓ 1. How to output something in Python
- ✓ 2. How to store and manipulate data in Python
- ✓ 3. How to take inputs in Python
- 4. If-Else statements

General syntax: If-elif-else



Example 1: Odd or even

Program that takes as input an integer, and prints “**even**” or “**odd**” depending on whether number is even or odd.

Example 1: Odd or even

Program that takes as input an integer, and prints “**even**” or “**odd**” depending on whether number is even or odd.

```
a = input("Enter number: ")  
b = int(a)
```


Example 1: Odd or even

Program that takes as input an integer, and prints “**even**” or “**odd**” depending on whether number is even or odd.

```
a = input("Enter number: ")
b = int(a)

if ( _____ ):
    print("number is even")
else:
    print("number is odd")
```

Example 1: Odd or even

Program that takes as input an integer, and prints “**even**” or “**odd**” depending on whether number is even or odd.

```
a = input("Enter number: ")
b = int(a)

if ( (b % 2) == 0 ):
    print("number is even")
else:
    print("number is odd")
```

Example 2: Equal, less than, or greater than

Write a program that takes as input two integers, and prints

- “equal” if the two numbers are equal,
- “greater than” if the first number is greater than the second,
- “less than” if the first number is smaller than the second.

Example 2: Equal, less than, or greater than

Write a program that takes as input two integers, and prints

- “equal” if the two numbers are equal,
- “greater than” if the first number is greater than the second,
- “less than” if the first number is smaller than the second.

```
a = input("First number:")
b = input("Second number:")
c = int(a)
d = int(b)

if ( _____ ):
    print("equal")

elif ( _____ ):
    print("greater than")

else:
    print("less than")
```

Example 2: Equal, less than, or greater than

Write a program that takes as input two integers, and prints

- “equal” if the two numbers are equal,
- “greater than” if the first number is greater than the second,
- “less than” if the first number is smaller than the second.

```
a = input("First number:")
b = input("Second number:")
c = int(a)
d = int(b)

if ( c == d ):
    print("equal")

elif ( ):
    print("greater than")

else:
    print("less than")
```

Example 2: Equal, less than, or greater than

Write a program that takes as input two integers, and prints

- “equal” if the two numbers are equal,
- “greater than” if the first number is greater than the second,
- “less than” if the first number is smaller than the second.

```
a = input("First number:")
b = input("Second number:")
c = int(a)
d = int(b)

if ( c == d ):
    print("equal")
elif ( c > d ):
    print("greater than")
else:
    print("less than")
```